SEMINAR · PHÂN TÍCH & THIẾT KẾ THUẬT TOÁN

Range Minimum Query
từ $O(n)$ xuống $O(1)$

Sparse Table, cây Cartesian, Fischer–Heun và câu hỏi thực tế: **khi nào dùng cấu trúc nào?**

Nhóm [Thành viên 1] · [Thành viên 2] · [Thành viên 3]

GVHD [Tên giảng viên]

Ngày [dd/mm/2026]

$$\mathrm{RMQ}(l, r) = \arg\min_{l \le i \le r} A[i]$$

RMQ(2, 7) → vị trí 5, giá trị 1

LỘ TRÌNH · 40 PHÚT

Năm phần, đi từ bài toán đến thực tế

01
Đặt vấn đề
Bài toán RMQ, biến thể tĩnh/động, cách đo chi phí $\langle P, Q \rangle$

0–4′

02
Cơ sở thuật toán
Duyệt, bảng mọi đoạn, chia khối, Segment Tree, Sparse Table + chứng minh

4–13′

03
Áp dụng giải bài toán
Chạy tay, demo tương tác, giải CSES, chọn cấu trúc theo điều kiện

13–19′

04
Tối ưu & độ phức tạp
Cây Cartesian, Fischer–Heun $\langle O(n), O(1) \rangle$, bitmask, cận dưới, benchmark tự đo

19–33′

05
Ứng dụng & bài tập
LCA, chuỗi (LCP/LCE), truy hồi tài liệu, xử lý ảnh, bài tập phân cấp

33–40′

01

Đặt vấn đề

Một câu hỏi rất đơn giản — nhưng được hỏi hàng triệu lần trên cùng một dữ liệu.

≈ 4 phút

ĐỘNG LỰC

Cùng một mảng — rất nhiều câu hỏi “nhỏ nhất trong đoạn”

CHUỖI THỜI GIAN

Nhiệt độ thấp nhất từ ngày l đến ngày r?

Dữ liệu cảm biến, giá cổ phiếu, log hệ thống: dữ liệu đã thu xong, câu hỏi đến liên tục.

CÂY

Tổ tiên chung gần nhất của u và v?

LCA quy về RMQ trên mảng độ sâu của Euler tour (Berkman–Vishkin; Bender–Farach-Colton).

CHUỖI / DNA

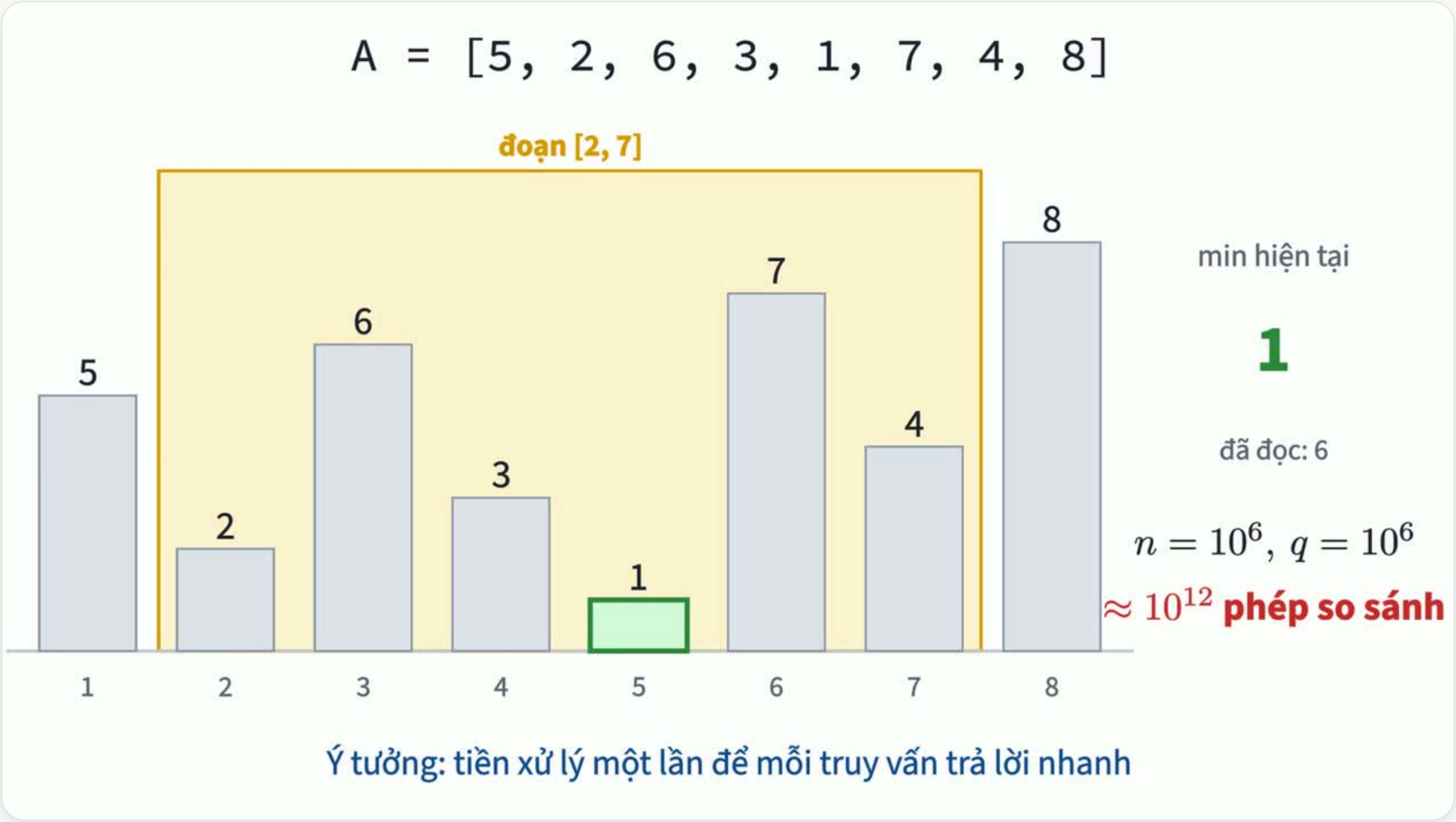
Hai hậu tố có chung tiền tố dài bao nhiêu?

LCE = RMQ trên mảng LCP của suffix array — lõi của so khớp xấp xỉ, chỉ mục nén.

Dữ liệu ít thay đổi, truy vấn rất nhiều ⇒ đáng để tiền xử lý một lần.

CÁCH NGÂY THƠ

Quét từng đoạn: đúng, nhưng lặp lại công việc



PHÁT BIỂU BÀI TOÁN

Range Minimum Query (RMQ)

Đầu vào: mảng $A[1..n]$ gồm các phần tử so sánh được; q truy vấn (l, r) với $1 \le l \le r \le n$.

$$\mathrm{RMQ}_A(l, r) = \arg\min_{l \le i \le r} A[i]$$

Trả về **vị trí** (argmin); giá trị là $A[\mathrm{RMQ}(l, r)]$. Nhiều vị trí bằng nhau → chọn **trái nhất**: so sánh cặp $(A[i], i)$.



$$\mathrm{RMQ}(2, 7) = 5, \quad \mathrm{RMQ}(1, 4) = 2, \quad \mathrm{RMQ}(6, 8) = 7$$

TĨNH VS ĐỘNG

Mảng cố định, hay xen kẽ **cập nhật** $A[i] \leftarrow x$?

TRỰC TUYẾN VS NGOẠI TUYẾN

Trả lời ngay từng truy vấn, hay biết trước cả lô?

DẠNG TRUY VẤN

Đoạn tùy ý, cửa sổ trượt cố định, hay hình chữ nhật (2D)?

ĐO CHI PHÍ THẾ NÀO?

Ký hiệu ⟨tiền xử lý, truy vấn⟩ và tổng chi phí

Ba con số cho mỗi cấu trúc

$P(n)$: thời gian tiền xử lý · $Q(n)$: thời gian một truy vấn · $S(n)$: bộ nhớ phụ.

$$T(q) = P(n) + q \cdot Q(n)$$

Viết gọn $\langle P(n), Q(n) \rangle$. Ví dụ duyệt trực tiếp là $\langle O(1), O(n) \rangle$. Mục tiêu tối thượng: $\langle O(n), O(1) \rangle$ — không thể tốt hơn vì phải đọc đầu vào.

Sao không làm như prefix sum?

Tổng: $\text{sum}(l, r) = P[r] - P[l-1]$. Phép **min không có phép "trừ"**.

MẢNG	PREFIX-MIN	MIN A[2..2]
X = [1, 5, 4]	[1, 1, 1]	5
Y = [1, 2, 4]	[1, 1, 1]	2

Cùng prefix-min nhưng đáp án khác ⇒ không suy ra được min đoạn từ hai giá trị prefix.

02

Cơ sở thuật toán

Từ hai thái cực đến Sparse Table — kèm chứng minh tính đúng và phân tích.

≈ 9 phút

HAI THÁI CỰC

Không tiền xử lý vs. lưu sẵn mọi đáp án

DUYỆT TRỰC TIẾP

$\langle O(1), O(n) \rangle$ bộ nhớ $O(1)$

Mỗi truy vấn quét $r - l + 1$ phần tử.

Tổng: $O(n \cdot q)$ — tệ khi q lớn.

+ Không tốn bộ nhớ, cập nhật $O(1)$, tốt khi đoạn ngắn.

BẢNG MỌI ĐOẠN

$\langle O(n^2), O(1) \rangle$ bộ nhớ $O(n^2)$

$$D[l][r] = \min\left(D[l][r-1],\ A[r]\right)$$

$n(n+1)/2$ ô, mỗi ô $O(1)$ nhờ quy hoạch động.

− $n = 10^6$: $5 \cdot 10^{11}$ ô ≈ 2 TB bộ nhớ.

> Có điểm nào ở giữa: tiền xử lý gần tuyến tính **và** truy vấn gần hằng số?

PHƯƠNG ÁN TRUNG GIAN 1

Chia khối √n: tóm tắt từng khối



min khối: 4 3 2 1

Truy vấn [3, 14] = quét lẻ **[3, 4]** + đọc min **khối 2, 3** + quét lẻ **[13, 14]** → min(6, 3, 2, 1) = 1

TIỀN XỬ LÝ

$O(n)$, bộ nhớ $O(\sqrt{n})$

TRUY VẤN

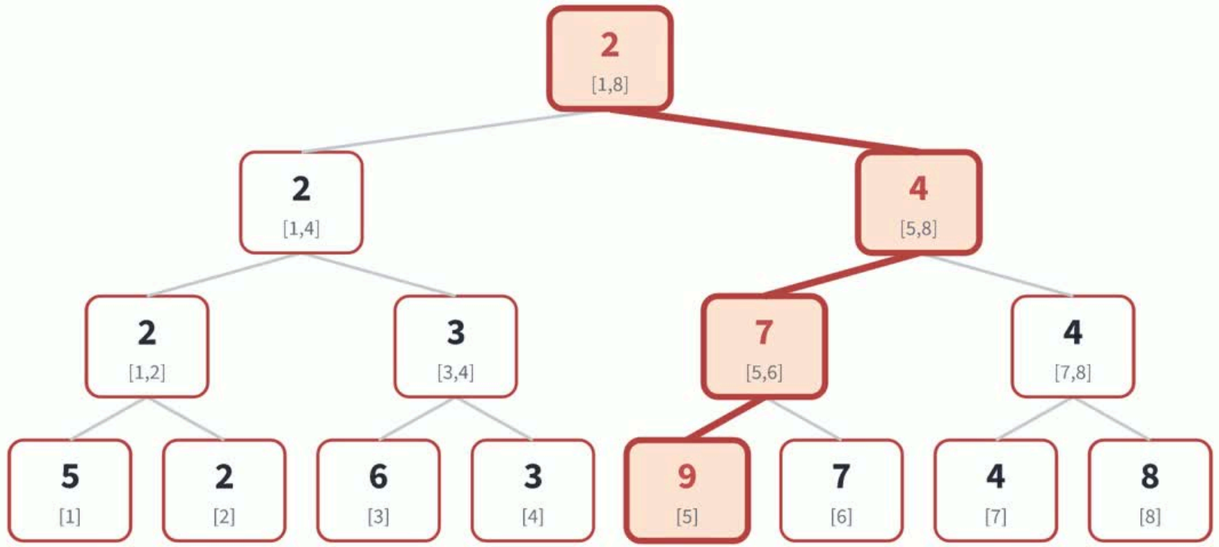
$\leq 2\sqrt{n} + \sqrt{n} = O(\sqrt{n})$

CẬP NHẬT ĐIỂM

quét lại 1 khối: $O(\sqrt{n})$

PHƯƠNG ÁN TRUNG GIAN 2

Segment Tree: chia đôi đệ quy, hỗ trợ cập nhật



$\log_2 n + 1$ nút ⇒ cập nhật $O(\log n)$. Sparse Table phải xây lại $O(n \log n)$

$\langle O(n), O(\log n) \rangle$

TRUY VẤN

Mỗi tầng chọn ≤ 2 nút ⇒ ≤ $2\lceil \log_2 n \rceil$ nút.

CẬP NHẬT ĐIỂM

Sửa lá + các tổ tiên: $O(\log n)$.

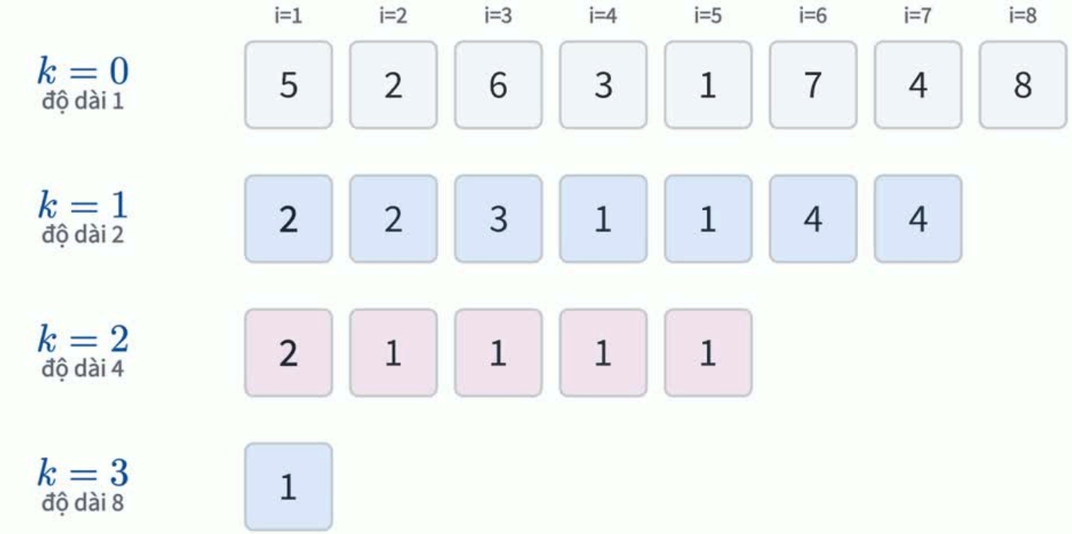
BỘ NHỚ

2n ô (bản lặp, bottom-up).

Ý TƯỞNG THEN CHỐT

Sparse Table: chỉ lưu các đoạn dài lũy thừa của 2

$$st[k][i] = \min\left(st[k-1][i],\ st[k-1][i+2^{k-1}]\right)$$



Mỗi ô $O(1)$, có $\approx n \log_2 n$ ô $\Rightarrow$ xây $O(n \log n)$, bộ nhớ $O(n \log n)$

$st[k][i]$ = min của đoạn dài 2^k bắt đầu tại i:

$$st[0][i] = A[i]$$
$$st[k][i] = \min\left(st[k-1][i],\ st[k-1][i+2^{k-1}]\right)$$

SỐ Ô

$\sum_{k=0}^{\lfloor \log_2 n \rfloor}(n - 2^k + 1) = \Theta(n \log n)$

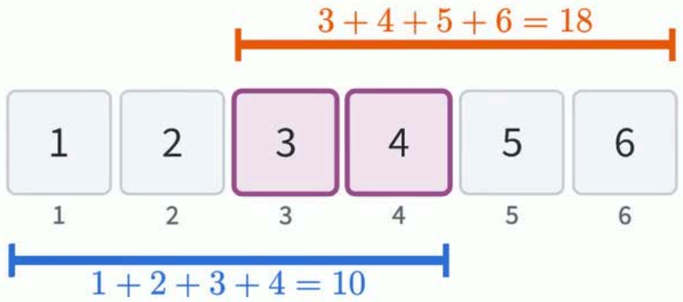
XÂY BẢNG

Mỗi ô $O(1) \Rightarrow \Theta(n \log n)$ thời gian và bộ nhớ.

TRUY VẤN $O(1)$

Hai đoạn chồng lấn phủ kín [l, r]

Nếu thay min bằng TỔNG: $B = [1, 2, 3, 4, 5, 6]$, đoạn $[1, 6]$



$$10 + 18 = 28 \neq 21 \quad ✗$$

Tổng: phần giao $3 + 4$ bị cộng hai lần ⇒ SAI. Dùng được: min, max, gcd, AND, OR

$$k = \lfloor \log_2(r - l + 1) \rfloor$$
$$\min\left(st[k][l],\ st[k][r - 2^k + 1]\right)$$

VÌ SAO ĐÚNG?

Lũy đẳng: $\min(x, x) = x$ — phần giao bị xét 2 lần vẫn không đổi kết quả.

KHÔNG DÙNG ĐƯỢC CHO

tổng, XOR, tích: phần giao bị tính 2 lần ⇒ cần Disjoint Sparse Table hoặc prefix.

CHỨNG MINH & PHÂN TÍCH

Tính đúng, thời gian và bộ nhớ của Sparse Table

Bổ đề phủ

Đặt $L = r - l + 1, k = \lfloor \log_2 L \rfloor \Rightarrow 2^k \le L < 2^{k+1}$.

- Cả hai đoạn $[l,\ l + 2^k - 1]$ và $[r - 2^k + 1,\ r]$ nằm trong $[l, r]$ vì $2^k \le L$.
- Hợp của chúng là $[l, r]$ vì $2 \cdot 2^k > L \Rightarrow$ không để hở.
- min có tính kết hợp + lũy đẳng ⇒ min(min đoạn 1, min đoạn 2) = min $[l, r]$. ∎

THỜI GIAN

Xây $\Theta(n \log n)$; truy vấn $O(1)$ nếu $\lfloor \log_2 \rfloor$ là $O(1)$: bảng `lg[]` $O(n)$ hoặc lệnh `__builtin_clz`.

BỘ NHỚ

$\Theta(n \log n)$ word. $n = 10^6$, int 32 bit: 20 tầng × 4 MB ≈ **80 MB**.

TỔNG CHO Q TRUY VẤN

$T(q) = O(n \log n + q)$. Cần argmin: lưu chỉ số, so sánh $(A[i], i)$.

CÀI ĐẶT

Sparse Table trong C++17 — 15 dòng

```cpp
struct SparseTable {                        // RMQ tĩnh: <O(n log n), O(1)>
    vector<vector<int>> st;
    SparseTable(const vector<int>& a) {
        int n = a.size(), K = __lg(n) + 1;
        st.assign(K, vector<int>(n));
        st[0] = a;                          // k = 0: đoạn dài 1
        for (int k = 1; k < K; ++k)
            for (int i = 0; i + (1 << k) <= n; ++i)
                st[k][i] = min(st[k-1][i], st[k-1][i + (1 << (k-1))]);
    }
    int query(int l, int r) const {         // đoạn đóng [l, r], 0-based
        int k = __lg(r - l + 1);            // ⌊log2 L⌋ bằng lệnh đếm bit
        return min(st[k][l], st[k][r - (1 << k) + 1]);
    }
};
```

DÒNG 4–10

Quy hoạch động theo k: hàng k dùng hàng k − 1.

DÒNG 11–14

`__lg` = vị trí bit cao nhất ⇒ $O(1)$, không dùng số thực.

ĐÃ KIỂM THỬ

Mã trên slide đã biên dịch và đối chiếu với duyệt trên 32 triệu đoạn.

03

Áp dụng giải bài toán

Chạy tay trên ví dụ, giải bài thi đấu, và chọn cấu trúc theo điều kiện đề bài.

≈ 6 phút

DEMO TƯƠNG TÁC

Sparse Table Explorer — chọn l, r hoặc bấm một ô



l 2 r 7

Mảng ví dụ

Ngẫu nhiên

độ dài L $7 - 2 + 1 = 6$

k = ⌊log₂ L⌋ $2\ (2k = 4)$

đoạn 1 $[2, 5] \to \mathrm{st}[2][2] = 1$

đoạn 2 $[4, 7] \to \mathrm{st}[2][4] = 1$

min = **1** tại vị trí 5

Duyệt: **6** lần đọc · Sparse Table: **2** lần tra bảng

BÀI TOÁN MẪU

CSES 1647 · Static Range Minimum Queries

ĐỀ BÀI (TÓM TẮT)

$n, q \le 2 \cdot 10^5$; mỗi truy vấn (a, b) in min của đoạn $[a, b]$. Không có cập nhật.

CÁCH	SỐ PHÉP TOÁN (XẤU NHẤT)	KẾT QUẢ
Duyệt	$n \cdot q = 4 \cdot 10^{10}$	**TLE**
Segment Tree	$n + q \cdot 2\log n \approx 7 \cdot 10^6$	**AC**
Sparse Table	$n \log n + 2q \approx 4 \cdot 10^6$	**AC**

```cpp
int main() {
    int n, q; scanf("%d %d", &n, &q);
    vector<int> a(n);
    for (int& x : a) scanf("%d", &x);
    SparseTable t(a);
    while (q--) {
        int l, r; scanf("%d %d", &l, &r);
        printf("%d\n", t.query(l - 1, r - 1));
    }
}
```

ĐỔI ĐỀ: CSES 1649 (CÓ CẬP NHẬT ĐIỂM)

Sparse Table phải xây lại $O(n \log n)$ mỗi lần ⇒ chuyển sang **Segment Tree** $O(\log n)$/thao tác.

CHỌN CẤU TRÚC THEO ĐIỀU KIỆN BÀI TOÁN

Sáu câu hỏi trước khi chọn cấu trúc dữ liệu

1 Có cập nhật xen kẽ truy vấn?	→ **Segment Tree** (lazy nếu cập nhật đoạn) · Fenwick 2 cây
2 Truy vấn là cửa sổ trượt độ dài cố định?	→ **Deque đơn điệu**: $O(n)$ cho mọi cửa sổ
3 Biết trước cả lô, và q ≪ n?	→ **Offline**: DSU (Arpa) hoặc co mảng (Alzamel et al.)
4 Rất ít truy vấn, hoặc đoạn rất ngắn?	→ **Duyệt trực tiếp**: không tốn build
5 n rất lớn, bộ nhớ hạn chế?	→ **RMQ tuyến tính** (bitmask) · succinct $2n + o(n)$ bit
6 Còn lại: tĩnh, nhiều truy vấn tùy ý	→ **Sparse Table**: đơn giản, $O(1)$

SO SÁNH VỚI CÁC CẤU TRÚC DỮ LIỆU KHÁC

Bảng tổng hợp: thời gian, cập nhật, bộ nhớ, phép toán

CẤU TRÚC	TIỀN XỬ LÝ	TRUY VẤN	CẬP NHẬT ĐIỂM	BỘ NHỚ PHỤ	PHÉP TOÁN
Duyệt trực tiếp	—	$O(n)$	$O(1)$	$O(1)$	mọi phép
Bảng mọi đoạn	$O(n^2)$	$O(1)$	$O(n^2)$	$O(n^2)$	kết hợp
Chia khối √n	$O(n)$	$O(\sqrt{n})$	$O(\sqrt{n})$	$O(\sqrt{n})$	kết hợp
Segment Tree	$O(n)$	$O(\log n)$	$O(\log n)$	$O(n)$	kết hợp (monoid)
Fenwick 2 cây (Dima–Ceterchi 2015)	$O(n)$	$O(\log n)$	$O(\log n)$*	$O(n)$	min / max
Sparse Table	$O(n \log n)$	$O(1)$	xây lại	$O(n \log n)$	lũy đẳng
Disjoint Sparse Table	$O(n \log n)$	$O(1)$	—	$O(n \log n)$	kết hợp
Fischer–Heun / bitmask	$O(n)$	$O(1)$	—	$O(n)$	min / max
Succinct (Fischer–Heun 2011)	$O(n)$	$O(1)$	—	$2n + o(n)$ bit	argmin

* trung bình (amortized) theo tác giả. Bộ nhớ tính bằng word (trừ dòng cuối). Disjoint Sparse Table dùng các đoạn không giao nhau nên đúng cho mọi phép kết hợp (tổng, tích ma trận…).

04

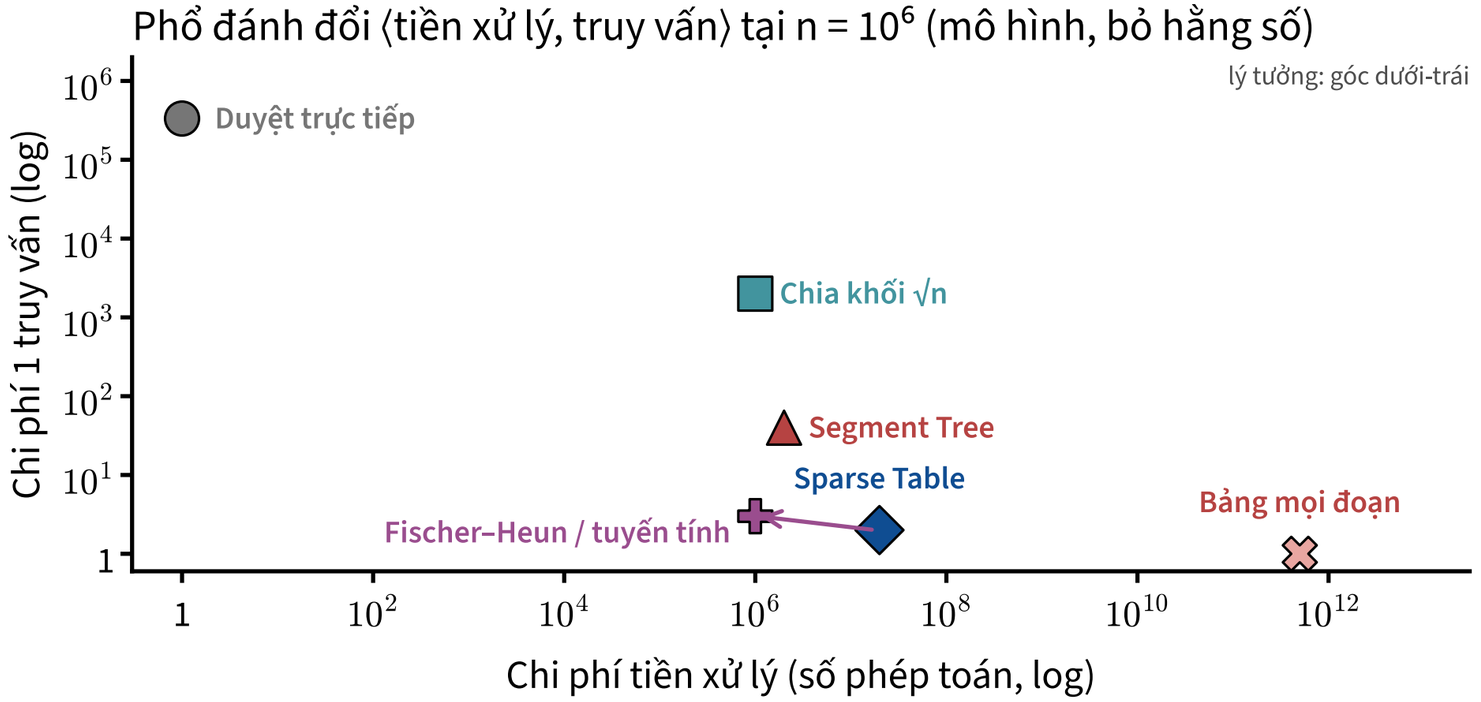
Tối ưu hóa & phân tích độ phức tạp

Bỏ thừa số log n: chia khối, cây Cartesian, Fischer–Heun, bitmask — rồi kiểm chứng bằng đo đạc.

≈ 14 phút

MỤC TIÊU

Có đạt được $\langle O(n), O(1) \rangle$ không?



KHÔNG THỂ TỐT HƠN

Phải đọc đủ n phần tử: $\Omega(n)$. Mỗi truy vấn: $\Omega(1)$.

SPARSE TABLE THỪA GÌ?

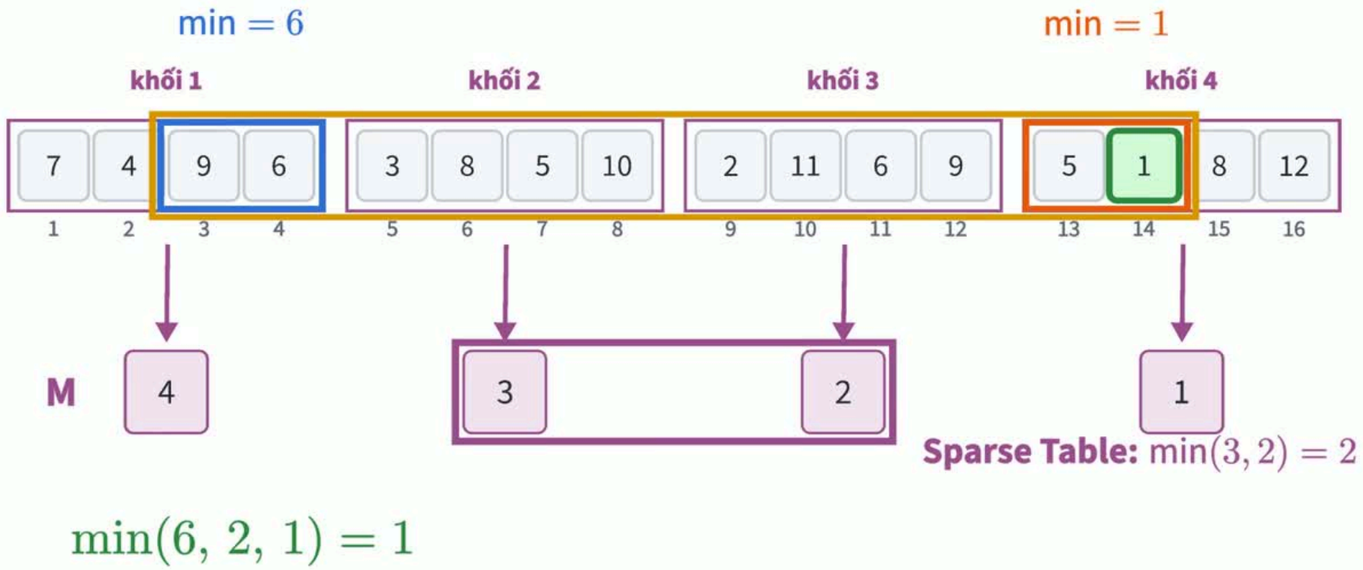
Một thừa số **log n** ở tiền xử lý và bộ nhớ.

CHIẾN LƯỢC

Chỉ dùng Sparse Table cho **n / log n** phần tử đại diện — phần còn lại xử lý bằng bảng tra nhỏ.

BƯỚC 1 · CHIA KHỐI HAI TẦNG

Tầng trên: Sparse Table trên min của các khối



Chọn $b = \Theta(\log n)$: tầng trên $O(n)$. Còn lại: truy vấn TRONG một khối, làm sao $O(1)$?

Khối kích thước $b = \Theta(\log n) \Rightarrow n/b$ khối:

$$\frac{n}{b} \log \frac{n}{b} \leq \frac{n}{b} \log n = O(n)$$

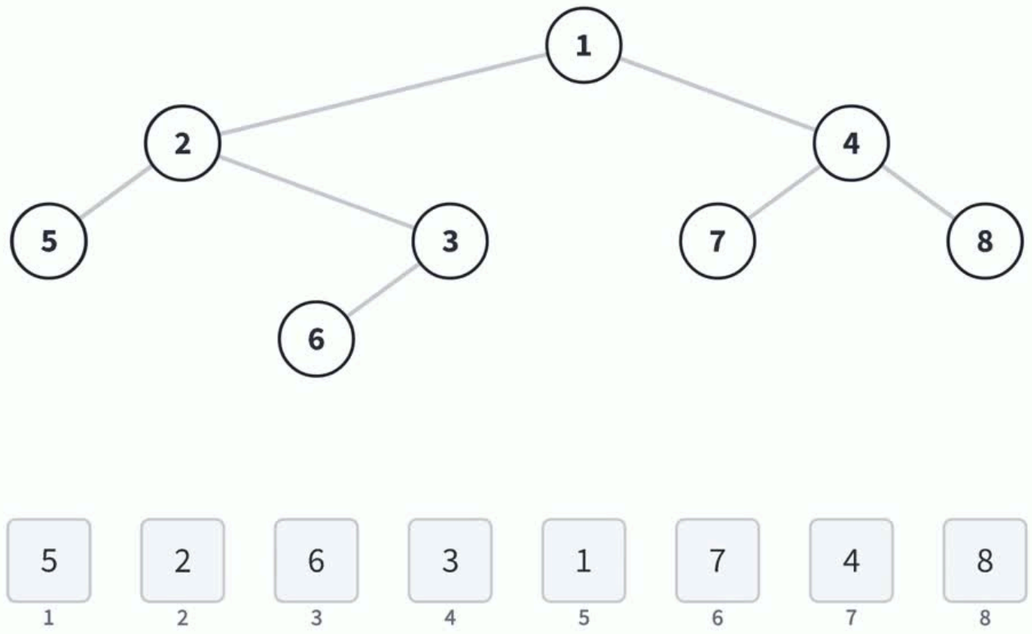
TRUY VẤN

hậu tố khối trái + Sparse Table giữa + tiền tố khối phải: 3 ứng viên.

CÒN THIẾU

Truy vấn **trong một khối** trong $O(1)$?

Đáp án RMQ chỉ phụ thuộc “hình dạng” của mảng



Xây bằng ngăn xếp: mỗi phần tử push/pop ≤ 1 lần $\Rightarrow O(n)$. RMQ $\equiv$ LCA

ĐỊNH NGHĨA (VUILLEMIN 1980)

Gốc = min; cây con trái/phải dựng đệ quy. Duyệt giữa = thứ tự chỉ số; cha nhỏ hơn con.

RMQ ≡ LCA

$\mathrm{RMQ}(l, r) = \mathrm{LCA}(l, r)$ trong cây Cartesian (Gabow–Bentley–Tarjan 1984).

XÂY $O(n)$

Ngăn xếp = nhánh phải; mỗi phần tử push/pop ≤ 1 lần.

BƯỚC 3 · FISCHER & HEUN (CPM 2006)

Khối cùng hình dạng cây ⇒ dùng chung bảng tra

b	1	2	3	4	5	6	7	8
Catalan C_b	1	2	5	14	42	132	429	1430
hoán vị $b!$	1	2	6	24	120	720	5040	40320

$$b = \tfrac{1}{4}\log_2 n \;\Rightarrow\; 4^b = \sqrt{n} \text{ loại khối}$$

bảng tra: $O(\sqrt{n}\cdot b^2) = o(n)$

$$\Rightarrow \langle O(n), O(1)\rangle$$

Tiền xử lý $O(n)$, truy vấn $O(1)$: tối ưu tiệm cận cho RMQ tĩnh

QUAN SÁT

Hai khối cùng hình dạng cây Cartesian có **cùng** bảng argmin cho mọi (l, r).

ĐẾM LOẠI KHỐI

Số hình dạng = số Catalan $C_b \le 4^b/b^{3/2}$ $\ll b!$.

CHỌN $b = \frac{1}{4}\log_2 n$

$4^b = \sqrt{n}$ loại; mỗi bảng b^2 ô ⇒ $O(\sqrt{n}\log^2 n) = o(n)$.

PHÂN TÍCH ĐỘ PHỨC TẠP

Fischer–Heun đạt $\langle O(n), O(1) \rangle$ — tính chi tiết

$$P(n) = \underbrace{O(n)}_{\text{min khối, loại khối}} + \underbrace{O\left(\tfrac{n}{b}\log\tfrac{n}{b}\right)}_{\text{Sparse Table tầng trên}} + \underbrace{O\left(4^b b^2\right)}_{\text{bảng trong khối}}$$

$$b = \tfrac{1}{4}\log_2 n \;\Rightarrow\; \tfrac{n}{b}\log n = 4n, \qquad 4^b b^2 = \sqrt{n}\cdot\tfrac{\log^2 n}{16} = o(n) \;\Rightarrow\; P(n) = O(n)$$

TRUY VẤN

≤ 2 lần tra bảng trong khối + 2 ô Sparse Table + 3 phép so sánh ⇒ $O(1)$.

BỘ NHỚ

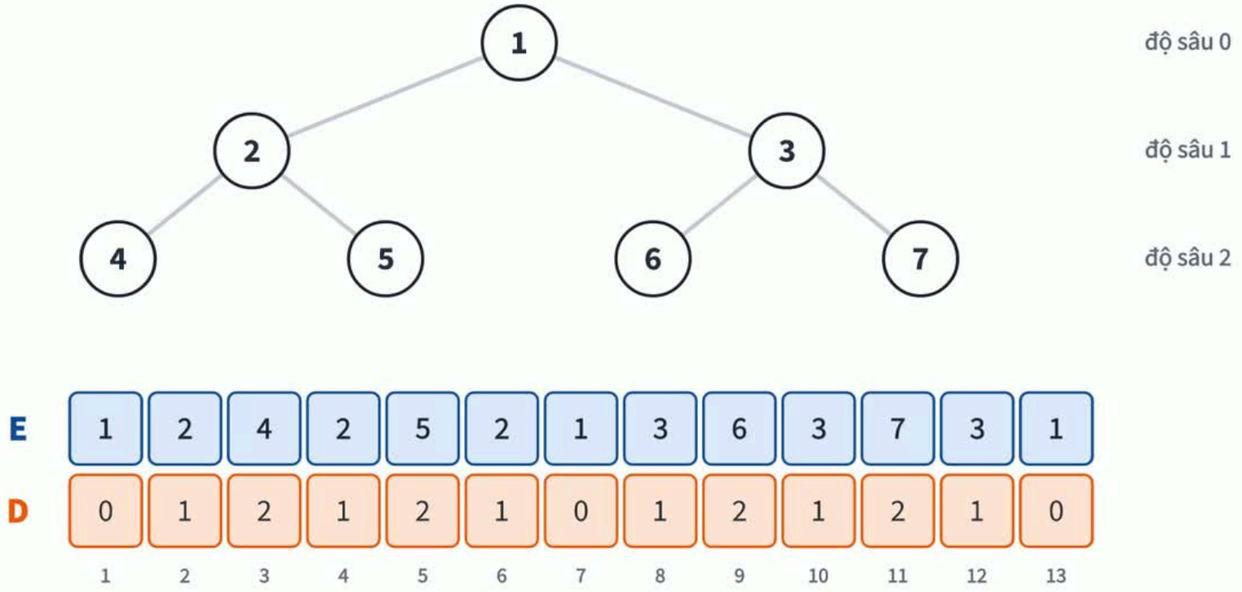
$O(n)$ word: loại của n/b khối + tầng trên $O(n)$ + bảng $o(n)$.

CÁI GIÁ

Hằng số lớn, cài đặt phức tạp ⇒ thực tế dùng biến thể đơn giản hơn.

CHIỀU NGƯỢC LẠI · LCA → RMQ

Euler tour biến LCA thành ±1 RMQ



D kề nhau chênh đúng ± 1 ⇒ bài toán ± 1 RMQ (Bender & Farach-Colton, 2000)

$$\mathrm{LCA}(u,v) = E\left[\arg\min D[first_u..first_v]\right]$$

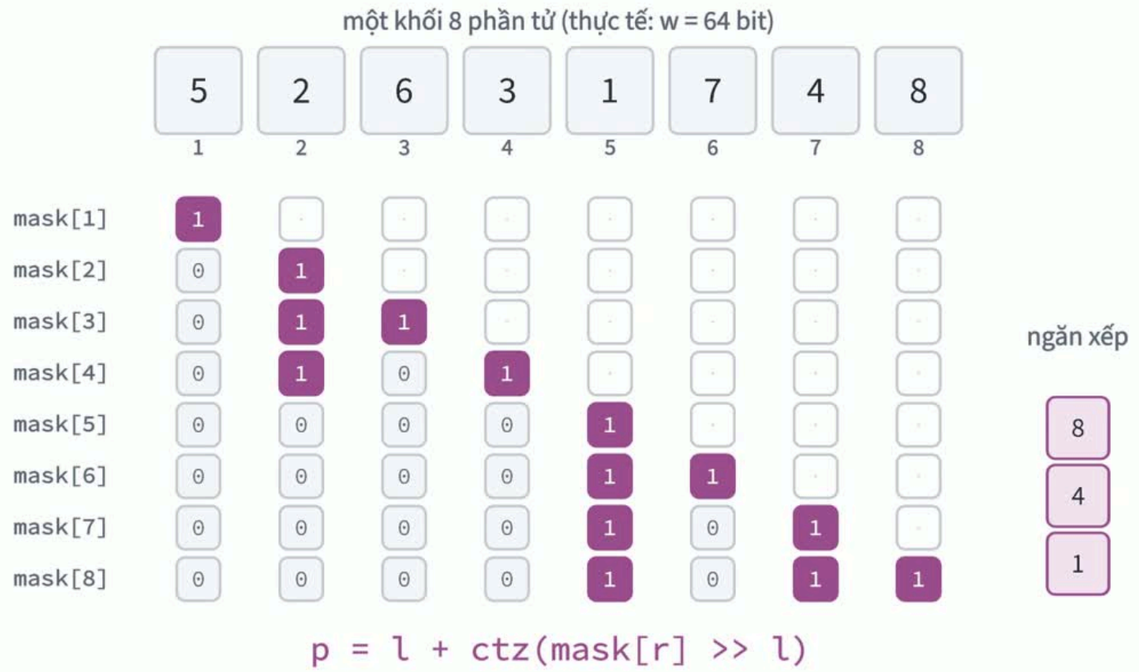
±1 RMQ

D kề nhau chênh ±1 ⇒ khối ½ log n chỉ có $2^{b-1} = O(\sqrt{n})$ kiểu.

GHI CÔNG ĐÚNG

Berkman–Vishkin et al. (1989–93); Bender & Farach-Colton (2000) trình bày lại đơn giản.

Khối = 1 từ máy 64 bit, truy vấn trong khối = 1 lệnh ctz



Kết hợp tầng trên (Sparse Table trên min các khối) $\Rightarrow \langle O(n), O(1) \rangle$ thực dụng

MASK[R]

Bit j = 1 ⇔ vị trí j còn trong ngăn xếp đơn điệu sau khi xét r (≡ nhánh phải cây Cartesian).

ĐỘ PHỨC TẠP

$O(n)$ mask + $\frac{n}{64} \log \frac{n}{64}$ ô Sparse Table; truy vấn $O(1)$.

NGUỒN GỐC

Kỹ thuật cộng đồng thi đấu: jcg (2019), brunomont (Codeforces blog 78931, 2020). Gần với block sparse table của Kowalski & Grabowski (2018).

CÀI ĐẶT RMQ TUYẾN TÍNH

Phần lõi: dựng mask và truy vấn trong khối

```
for (int s = 0; s < n; s += 64) {          // từng khối 64 phần tử
    uint64_t cur = 0;                       // ngăn xếp đơn điệu = bitmask
    for (int i = s; i < min(n, s + 64); ++i) {
        while (cur && a[s + 63 - __builtin_clzll(cur)] > a[i])
            cur ^= 1ULL << (63 - __builtin_clzll(cur));   // pop đỉnh
        cur |= 1ULL << (i - s);                          // push i
        mask[i] = cur;
    }
}
int inBlock(int l, int r) {                 // l, r cùng khối
    uint64_t m = mask[r] & (~0ULL << (l & 63)); // bỏ bit < l
    return a[(l & ~63) + __builtin_ctzll(m)];    // bit 1 thấp nhất
}
int query(int l, int r) {
    int bl = l >> 6, br = r >> 6;
    if (bl == br) return inBlock(l, r);
    int ans = min(inBlock(l, bl * 64 + 63), inBlock(br * 64, r));
    if (bl + 1 < br) ans = min(ans, top.query(bl + 1, br - 1));
    return ans;                             // top = Sparse Table trên min khối
}
```

TIE-BREAK

Chỉ pop khi **>** (không ≥) ⇒ trả về vị trí trái nhất.

VÌ SAO ĐÚNG?

Vị trí min trái nhất p của [l, r] không bị pop bởi phần tử nào trong (p, r]; mọi phần tử còn lại trong [l, p) sẽ ≤ A[p] — mâu thuẫn.

ĐÃ KIỂM THỬ

Mã trên slide đã đối chiếu với duyệt trên 32 triệu đoạn.

GIỚI HẠN DƯỚI & BỘ NHỚ NÉN

Cần ít nhất bao nhiêu bit? Có nhanh hơn được không?

KHÔNG GIỮ MẢNG A (ENCODING)

Từ các câu trả lời RMQ dựng lại được cây Cartesian ⇒ cần $\geq \log_2 C_n = 2n - \Theta(\log n)$ bit.

Đạt được: $2n + o(n)$ **bit**, $O(1)$ — Fischer (LATIN 2010) / Fischer & Heun (SICOMP 2011). Trước đó: Sadakane $4n + o(n)$ (2007).

Cell-probe: thời gian t ⇒ cần $2n + n/(\log n)^{\tilde{O}(t^2)}$ bit (Liu & Yu, STOC 2020).

ĐƯỢC ĐỌC A (INDEXING)

Thêm $O(n/c)$ bit ⇒ truy vấn $\Omega(c)$, và đạt được (Brodal–Davoodi–Rao 2010/12).

RMQ ĐỘNG

$\Theta(\log n / \log\log n)$ mỗi thao tác (Alstrup–Husfeldt–Rauhe 1998; Brodal–Davoodi–Rao 2011) ⇒ Segment Tree gần tối ưu.

> "$O(n)$ bộ nhớ" = $O(n)$ **word** = $O(n \log n)$ bit — khác 2n bit.

LỊCH SỬ · 45 NĂM CỦA MỘT BÀI TOÁN "ĐƠN GIẢN"

Các mốc chính (đã kiểm chứng nguồn)

1980

Vuillemin

Cây Cartesian (CACM)

1984

Gabow–Bentley–Tarjan · Harel–Tarjan

RMQ → LCA; LCA $O(1)$

$\langle O(n), O(1) \rangle$ đầu tiên

1989–93

Berkman–Vishkin et al.

Euler tour → ±1 RMQ, thuật toán song song

2000

Bender & Farach-Colton

Trình bày lại đơn giản: Sparse Table + khối

2006–11

Fischer & Heun

RMQ trực tiếp; succinct $2n + o(n)$ bit

tối ưu bit

2017–26

Thực nghiệm & phần cứng

Ferrada–Navarro, Baumstark et al. (2017); Liu–Yu cận dưới (2020); RTXRMQ GPU (2024)

PHÂN TÍCH TỔNG THỂ · N = $2^{20} \approx 10^6$

Thời gian và bộ nhớ: lý thuyết đặt cạnh số đo

CẤU TRÚC	$\langle P, Q \rangle$	BỘ NHỚ PHỤ (ĐO)	BUILD (ĐO)	NS / TRUY VẤN (ĐO)	T(Q = 10^6) ĐO
Duyệt trực tiếp	$\langle 1, n \rangle$	0	0	21 402	≈ 21 s*
Chia khối √n	$\langle n, \sqrt{n} \rangle$	4 KiB	1,4 ms	107	108 ms
Segment Tree	$\langle n, \log n \rangle$	8,0 MiB	1,1 ms	101	107 ms
RMQ tuyến tính	$\langle n, 1 \rangle$	8,8 MiB	8,0 ms	18,2	**26 ms**
Sparse Table	$\langle n \log n, 1 \rangle$	76 MiB	31 ms	9,8	41 ms

Sparse Table: truy vấn nhanh nhất nhưng tốn **8,6×** bộ nhớ và **3,9×** thời gian build so với RMQ tuyến tính.

Ở q = 10^6, RMQ tuyến tính thắng về **tổng** thời gian — đúng như lý thuyết dự đoán.

Số đo tự chạy (i5-13500HX, GCC 15.3 -O3, median 5 lần). * ước lượng P + q·Q từ số đo, không chạy trực tiếp.

BENCHMARK TỰ ĐO · PHƯƠNG PHÁP

Đo thế nào để kết luận có căn cứ?

MÔI TRƯỜNG

- Intel i5-13500HX, ghim 1 P-core
- GCC 15.3, `-O3 -march=native`
- L3 24 MiB, RAM 15 GiB
- Laptop, governor powersave

THIẾT KẾ THÍ NGHIỆM

- 5 cấu trúc, cùng dữ liệu & truy vấn
- Quét n: $2^{10} \rightarrow 2^{23}$
- Quét độ dài đoạn L, số truy vấn q, tỉ lệ cập nhật
- 5 lần, median + IQR, đảo thứ tự chạy

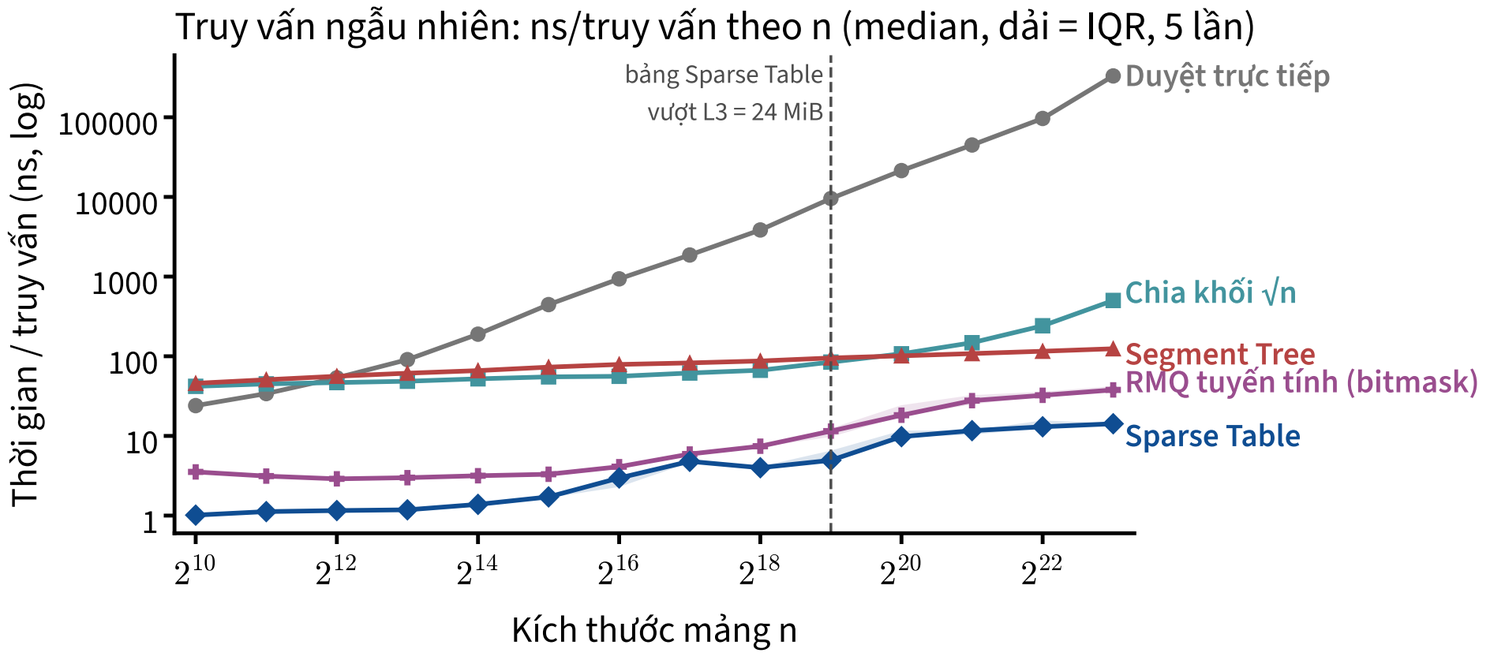
TÍNH ĐÚNG

- Self-test: 36,4 triệu phép đối chiếu với duyệt
- Checksum mọi lượt đo khớp nhau
- Bộ nhớ = payload mảng phụ (không phải RSS)
- Mã + CSV kèm theo để tái lập

> Đây là **microbenchmark** trên một máy: so sánh xu hướng, không phải bảng xếp hạng tuyệt đối. Không trộn với số liệu trong paper.

BENCHMARK TỰ ĐO · TRUY VẤN THEO N

Big-O dự đoán đúng xu hướng — bộ nhớ đệm quyết định hằng số

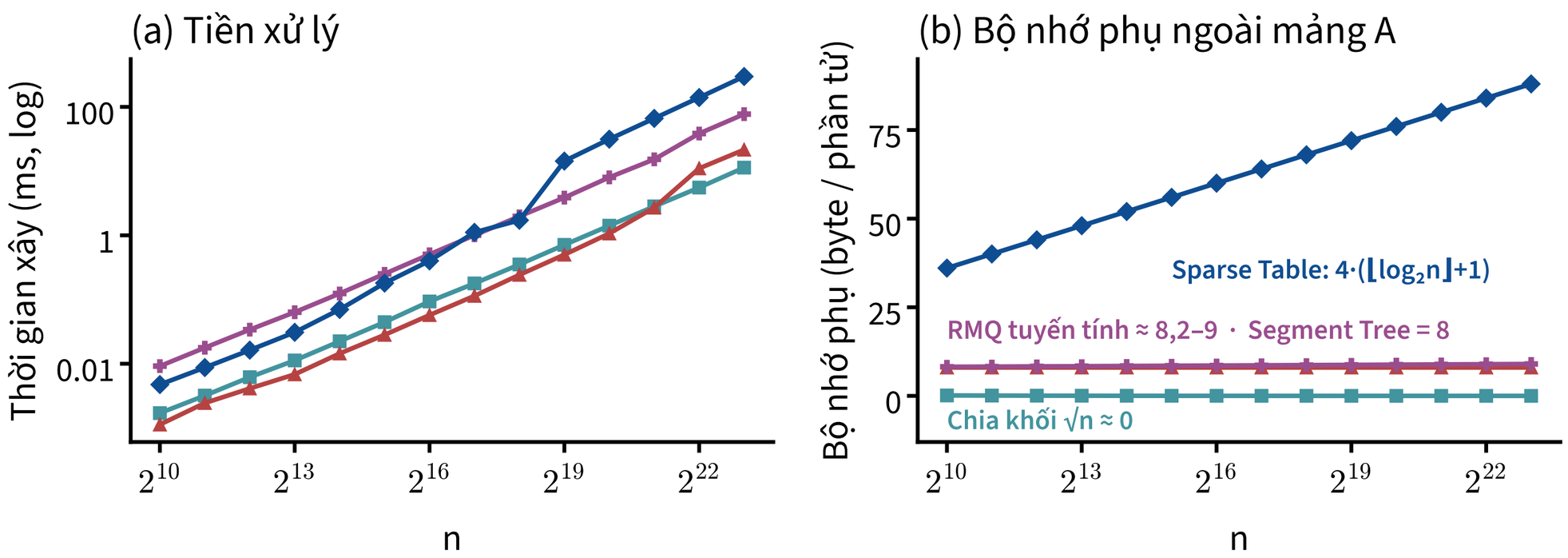


Sparse Table: 1 ns → 14 ns; nhảy bậc khi bảng **vượt L3** ($n \approx 2^{19}$).

RMQ tuyến tính: $O(1)$ nhưng 2 lần truy cập phụ thuộc nhau (mask → A) ⇒ chậm hơn ~2–3×.

$n \le 2^{12}$: duyệt thẳng nhanh hơn Segment Tree (SIMD, không rẽ nhánh).

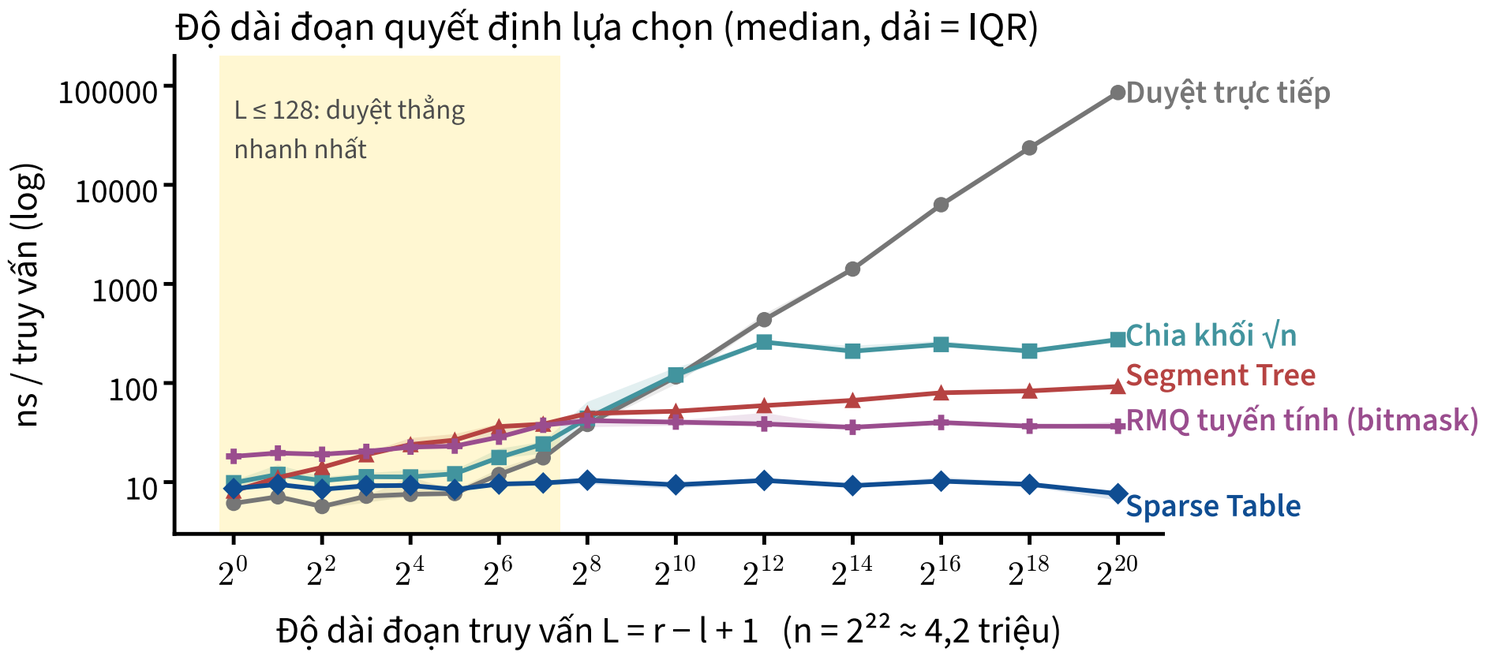
Cái giá của $O(1)$: thời gian xây và bộ nhớ



$n = 2^{23}$: Sparse Table build **297 ms**, 88 byte/phần tử · RMQ tuyến tính **77 ms**, 9 byte/phần tử · Segment Tree 22 ms, 8 byte/phần tử.

BENCHMARK TỰ ĐO · ĐỘ DÀI ĐOẠN

Đoạn ngắn: duyệt thắng thắng mọi cấu trúc



L ≤ 128: duyệt ≈ 6–18 ns, không cần tiền xử lý.

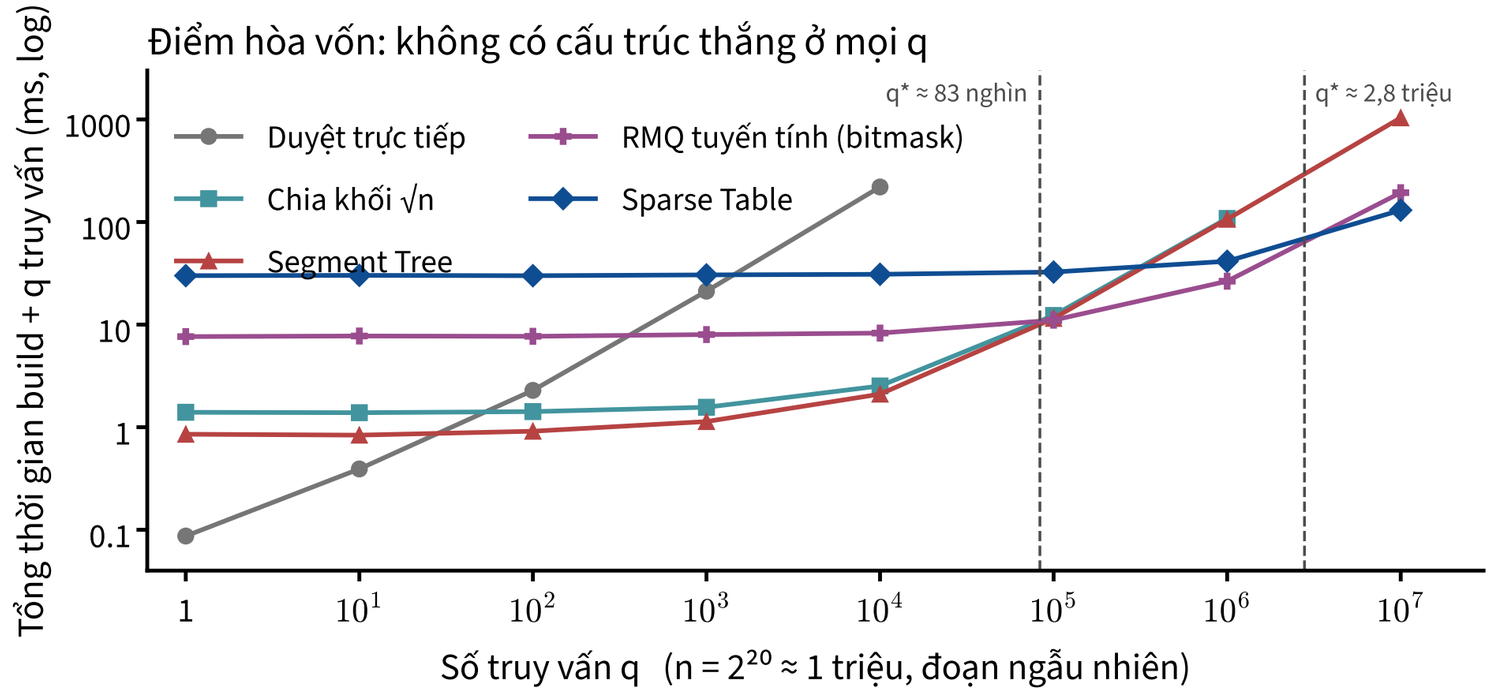
Sparse Table phẳng ≈ 8–10 ns với mọi L — đúng nghĩa $O(1)$.

Segment Tree tăng theo log L (8 → 92 ns).

Không dùng kết quả này để phủ nhận $O(n)$ xấu nhất của duyệt.

BENCHMARK TỰ ĐO · ĐIỂM HÒA VỐN

Query $O(1)$ chưa chắc tổng thời gian nhỏ nhất



Mô hình $T(q) \approx p + q\,t \Rightarrow$

$$q^* = \frac{p_A - p_B}{t_B - t_A}$$

$q \lesssim 8{\cdot}10^4$: Segment Tree / chia khối (build rẻ).

$8{\cdot}10^4 \lesssim q \lesssim 2{,}8{\cdot}10^6$: RMQ tuyến tính.

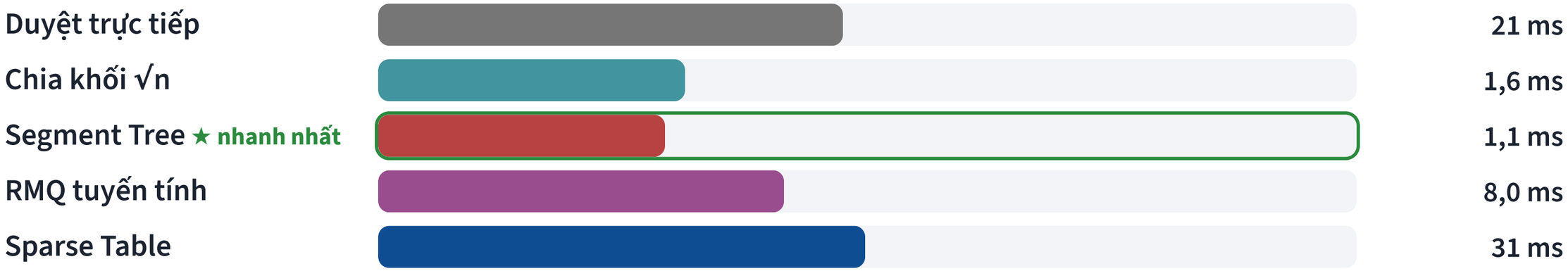
$q \gtrsim 2{,}8{\cdot}10^6$: Sparse Table bù được chi phí build.

DEMO TƯƠNG TÁC · SỐ ĐO THẬT, N = 2^{20}

Kéo q và xem cấu trúc nào thắng

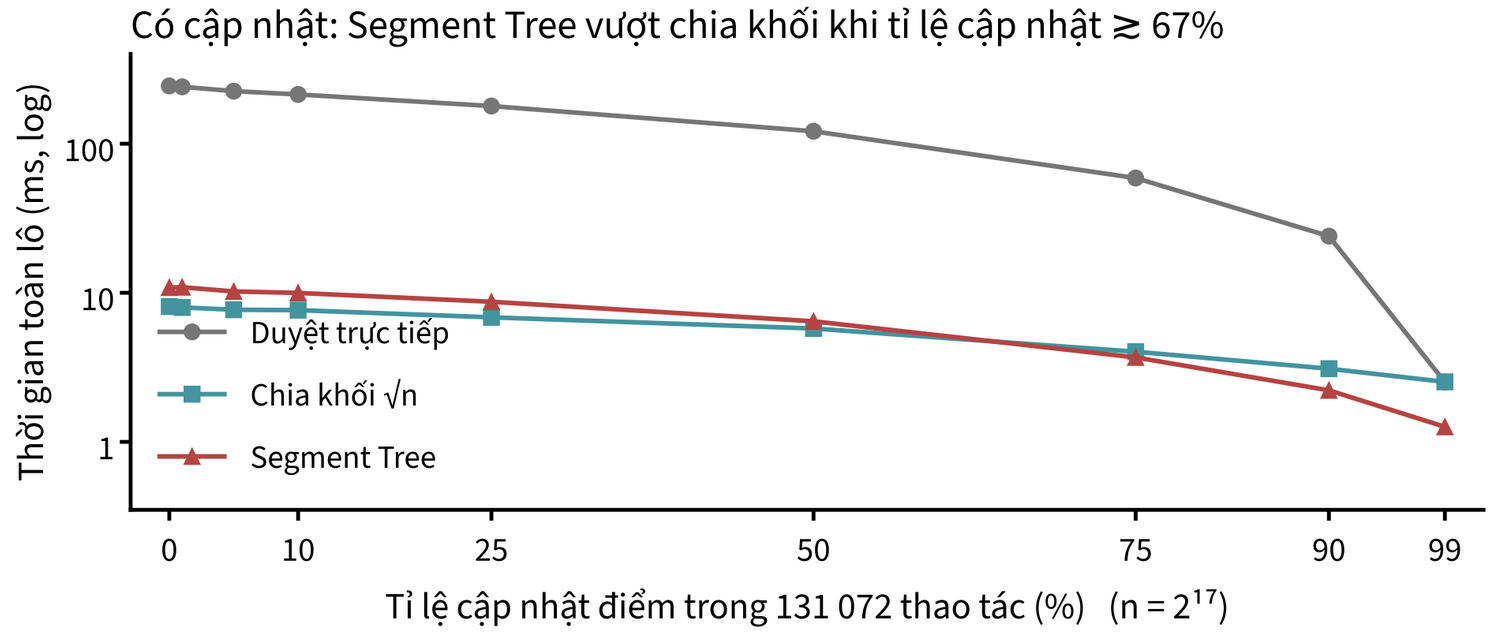
Số truy vấn q = **10^3**

thanh sọc = ước lượng p + q·t (không đo vì quá chậm) · trục log



BENCHMARK TỰ ĐO · CÓ CẬP NHẬT

Có cập nhật: cấu trúc tĩnh bị loại, Segment Tree lên ngôi



Ít cập nhật: chia khối nhanh hơn (quét liên tục, SIMD).

≳ 67% cập nhật: Segment Tree thắng — cập nhật $O(\log n)$ so với quét lại khối $O(\sqrt{n})$.

Sparse Table / RMQ tuyến tính: mỗi cập nhật phải xây lại ⇒ không đưa vào.

ĐỐI CHIẾU SỐ LIỆU ĐÃ CÔNG BỐ

Nghiên cứu khác đo được gì? (số của tác giả, máy của họ)

NGUỒN	THIẾT LẬP	KẾT QUẢ CHÍNH
brunomont 2020 · Codeforces	khối 64, 10^6 truy vấn	$n = 10^6$: bitmask 44 ns · Sparse Table 18 · Sqrt Tree 51 · Segment Tree 138 ns. Build $n = 10^7$: 140 vs 515 ms
Kowalski & Grabowski 2018	$n = 10^9$, i7-4790	Block Sparse Table: build 0,6 s, 34,6 bit/phần tử (gồm A) · Sparse Table đầy đủ: 436,6 s, 404,9 bit · SDSL: 23,9 s, 2,54 bit
Ferrada & Navarro 2017	$n = 10^4 \ldots 10^{10}$	≈ 2,1 bit/phần tử, 1–3 µs/truy vấn; nhanh hơn SDSL 3–6×
Baumstark et al. 2017	Xeon E5-4640	≈ 2,16 bit/phần tử; truy vấn nhanh hơn tới 3× so với cài đặt trước
Dima & Ceterchi 2015	$n = 10^5$, 10^7 thao tác	Fenwick 2 cây: truy vấn 0,551 s vs Segment Tree 2,397 s; cập nhật 0,672 vs 1,274 s
RTXRMQ 2024 · GPU	GPU RTX 6000 Ada	≈ 5 ns/RMQ (đoạn lớn, lô 2^{26}); tới 5× so với CPU 192 nhân

Xu hướng khớp số đo của nhóm: Sparse Table truy vấn nhanh nhất; bitmask/block build nhanh, nhỏ hơn nhiều.

BÀI HỌC TỪ BENCHMARK

Bốn kết luận có căn cứ

1 · Query $O(1)$ ≠ tổng nhanh nhất

Luôn tính $T(q) = P + qQ$; điểm hòa vốn phụ thuộc q.

2 · Bộ nhớ là chi phí thật

Sparse Table 76–88 byte/phần tử ⇒ rơi khỏi cache, build chậm.

3 · Workload đổi người thắng

Đoạn ngắn → duyệt; nhiều cập nhật → Segment Tree.

4 · Big-O cho xu hướng, đo cho quyết định

Hằng số, cache, rẽ nhánh, SIMD không có trong Big-O.

05

Ứng dụng thực tiễn & bài tập

RMQ là “viên gạch” bên trong LCA, chỉ mục chuỗi, truy hồi tài liệu, xử lý ảnh…

≈ 6 phút

MỞ RỘNG THUẬT TOÁN

Khi bài toán thay đổi một chút

CẬP NHẬT ĐOẠN

Lazy Segment Tree: cộng/gán cả đoạn, $O(\log n)$. Ví dụ: +3 vào [2,5] ⇒ min[2,7] = 4.

CỬA SỔ TRƯỢT

Deque đơn điệu $O(n)$; bộ lọc min/max ảnh (van Herk / Gil–Werman), streaming (Lemire 2006).

NGOẠI TUYẾN

Arpa's trick (DSU) $O((n+q)\cdot\alpha(n))$; lô nhỏ q ≪ n: co mảng còn $O(q)$ (Alzamel et al. 2017).

RMQ 2D

Sparse Table 2D: 4 hình chữ nhật chồng lấn. Không có "cây Cartesian 2D" (Demaine–Landau–Weimann).

PHÉP TOÁN KHÁC

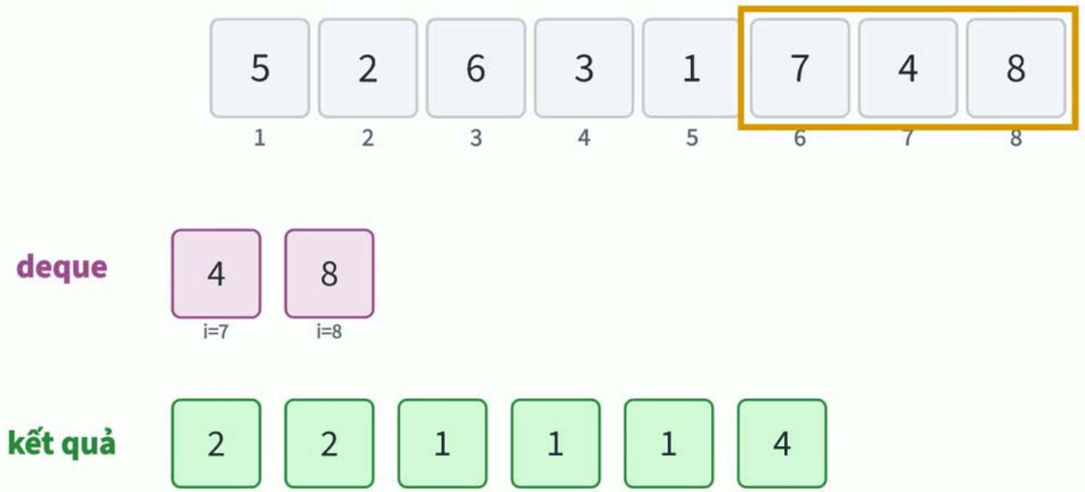
Disjoint Sparse Table / Sqrt Tree: $O(1)$ cho mọi phép kết hợp (tổng, tích ma trận…).

SONG SONG & GPU

Berkman–Vishkin (PRAM); RTXRMQ dùng nhân ray tracing (2024); GPU-RMQ (2026).

TRƯỜNG HỢP ĐẶC BIỆT · CỬA SỔ TRƯỢT

Min của mọi cửa sổ độ dài k trong $O(n)$



Mỗi chỉ số vào/ra deque ≤ 1 lần $\Rightarrow O(n)$ cho cả $n-k+1$ cửa sổ

Ý TƯỞNG

Deque chỉ số, giá trị tăng dần; bỏ cuối khi lớn hơn phần tử mới, bỏ đầu khi hết hạn.

SO SÁNH

Deque $O(n)$ tổng, $O(k)$ bộ nhớ — nhưng không trả lời được đoạn tùy ý về sau như Sparse Table.

ỨNG DỤNG

Lọc min/max ảnh (erosion), giám sát chuỗi thời gian.

ỨNG DỤNG 1 · CÂY

LCA và khoảng cách trên cây: so sánh các lời giải

PHƯƠNG PHÁP	TIỀN XỬ LÝ	MỖI LCA	BỘ NHỚ	KHI NÀO CÓ LỢI
Leo cha, cân bằng độ sâu	$O(n)$	$O(h)$	$O(n)$	rất ít truy vấn
Binary lifting	$O(n \log n)$	$O(\log n)$	$O(n \log n)$	cần thêm tổ tiên thứ k, gộp min/max trên đường
Euler tour + Sparse Table	$O(n \log n)$	$O(1)$	$O(n \log n)$	rất nhiều LCA trên cây tĩnh
Euler tour + ±1 RMQ	$O(n)$	$O(1)$	$O(n)$	tối ưu tiệm cận (Bender–Farach-Colton)

KHOẢNG CÁCH

$\mathrm{dist}(u, v) = d_u + d_v - 2\, d_{\mathrm{LCA}(u,v)}$ — CSES 1135, 1688.

SINH HỌC TÍNH TOÁN

Ánh xạ LCA cây gen → cây loài để suy ra sự kiện nhân đôi gen (Zmasek & Eddy, Bioinformatics 2001).

Suffix array + LCP + RMQ = LCE trong $O(1)$

	b	a	n	a	n	a
	1	2	3	4	5	6

rank	SA	hậu tố	LCP
1	6	a	0
2	4	a n a	1
3	2	a n a n a	3
4	1	b a n a n a	0
5	5	n a	0
6	3	n a n a	2

$\text{LCE}(i, j) = \min \text{LCP}[\text{rank}_i + 1..\text{rank}_j]$ — một truy vấn RMQ, $O(1)$ sau tiền xử lý

$$\text{LCE}(i, j) = \min_{\text{rank}_i < t \le \text{rank}_j} \text{LCP}[t]$$

DÙNG Ở ĐÂU

So khớp k sai khác (Landau–Vishkin); cây hậu tố nén (CST++, SDSL `cst_sct3` : `lca` gọi `rmq`).

LƯU Ý

$O(1)$ là SAU khi đã xây SA, LCP, RMQ. So trực tiếp có thể nhanh hơn khi chuỗi khác nhau sớm (Ilie–Navarro–Tinta 2010).

ỨNG DỤNG 3 · TRUY HỒI TÀI LIỆU

Liệt kê tài liệu chứa mẫu P (Muthukrishnan 2002)

- Các lần xuất hiện của P = một khoảng $[sp, ep]$ trên suffix array của cả tập tài liệu.
- $C[i]$: tài liệu chứa hậu tố thứ i; $L[i]$: vị trí trước đó cùng tài liệu (0 nếu chưa có).
- $p = \mathrm{RMQ}_L(i, j)$. Nếu $L[p] < sp$: báo $C[p]$ (lần đầu xuất hiện), đệ quy hai bên; ngược lại dừng.
- Mỗi tài liệu báo đúng 1 lần ⇒ $O(m + docc)$, không phụ thuộc số lần xuất hiện.

VÍ DỤ (SP = 1)

I	1	2	3	4	5	6
C	2	1	2	3	1	3
L	**0**	**0**	1	**0**	2	4

RMQ chọn vị trí 1, 2, 4 (L = 0 < sp) ⇒ báo {2, 1, 3}; các nhánh còn lại có L ≥ sp nên dừng.

Bản lập trình thi đấu: liệt kê giá trị phân biệt trong a[l..r] với prev[i].

ỨNG DỤNG KHÁC · PHÂN BIỆT “DÙNG THẬT” VÀ “HỌ HÀNG”

RMQ trong các lĩnh vực khác

CƠ SỞ DỮ LIỆU · DÙNG THẬT

Truy vấn MAX trên khối dữ liệu OLAP bằng cấu trúc tiền tính phân cấp (Ho, Agrawal, Megiddo, Srikant — SIGMOD 1997).

XỬ LÝ ẢNH · HỌ HÀNG

Erosion/dilation = lọc min/max cửa sổ; van Herk (1992), Gil & Werman (IEEE TPAMI 1993): ~3 phép so sánh/điểm ảnh.

CHUỖI THỜI GIAN · HỌ HÀNG

Bộ lọc max-min trực tuyến ≤ 3 phép so sánh/phần tử (Lemire 2006), được GNU Scientific Library trích dẫn.

THUẬT TOÁN · DÙNG THẬT

Hình chữ nhật lớn nhất trong histogram: chia để trị theo RMQ, hoặc ngăn xếp $O(n)$ (LeetCode 84).

PHẢN VÍ DỤ · DÙNG RMQ SAI CHỖ

Euler tour + RMQ cho LCA, nhưng KHÔNG cho min trên đường đi

Cây: 1–2, 1–3, 2–4, 2–5, 3–6, 3–7. Trọng số đỉnh: $w_1 = 10$, $w_2 = 8$, $w_3 = 6$, $w_4 = 7$, **$w_5 = -100$**.

Đường 4 → 3 = 4, 2, 1, 3 ⇒ min đúng = **6**.

Đoạn Euler giữa first[4] và first[3]: `4, 2, 5, 2, 1, 3` — chứa đỉnh 5 **ngoài đường đi** ⇒ RMQ trên trọng số trả **−100**.

ĐÚNG

RMQ trên **độ sâu** D của Euler tour ⇒ LCA.

TRỌNG SỐ TĨNH

Binary lifting kèm min: $O(n \log n)$ / $O(\log n)$ mỗi truy vấn.

CÓ CẬP NHẬT

Heavy-Light Decomposition + Segment Tree: $O(\log^2 n)$.

BÀI TẬP ỨNG DỤNG · TỪ DỄ ĐẾN KHÓ

Lộ trình luyện tập (đã kiểm tra đường dẫn)

Mức 1 · Trực tiếp

- CSES 1647 — Sparse Table
- CSES 1649 — Segment Tree
- VNOJ NKLINEUP — max − min

Mức 2 · Họ hàng

- VNOJ MINK — deque
- CSES 1645 — ngăn xếp
- VNOJ QMAX2 — lazy

Mức 3 · Kết hợp

- CF 689D — ST + chặt nhị phân
- CF 1454F — ST + chặt nhị phân
- CF 475D — Sparse Table gcd

Mức 4 · Cây

- CSES 1688 — Euler + RMQ
- CSES 1135 — khoảng cách
- VNOJ LUBENICA — phản ví dụ

Mức 5 · Chuỗi

- ABC213 F — SA + LCP
- CF 1073G — SA + LCP + RMQ

Đọc thêm (tiếng Việt)

- VNOI Wiki · RMQ
- VNOI Wiki · LCA
- VNOI Wiki · deque min-max

KẾT LUẬN

Bốn điều mang về

Tiền xử lý để tái sử dụng

RMQ là phổ đánh đổi $\langle P, Q, S \rangle$: từ $\langle O(1), O(n) \rangle$ đến $\langle O(n), O(1) \rangle$.

Sparse Table = lũy đẳng

Hai đoạn chồng lấn ⇒ $O(1)$; mặc định cho RMQ tĩnh.

Tối ưu nhờ cây Cartesian

Chia khối + chung bảng theo hình dạng ⇒ $\langle O(n), O(1) \rangle$; $2n + o(n)$ bit là giới hạn thông tin.

Chọn theo workload, có đo đạc

Số truy vấn, độ dài đoạn, cập nhật, bộ nhớ quyết định lựa chọn.

Các công trình nền tảng

[1] J. Vuillemin. A unifying look at data structures. CACM 23(4):229–239, 1980.

[2] H. N. Gabow, J. L. Bentley, R. E. Tarjan. Scaling and related techniques for geometry problems. STOC 1984, 135–143.

[3] D. Harel, R. E. Tarjan. Fast algorithms for finding nearest common ancestors. SIAM J. Comput. 13(2):338–355, 1984.

[4] O. Berkman, U. Vishkin. Recursive star-tree parallel data structure. SIAM J. Comput. 22(2):221–242, 1993.

[5] M. A. Bender, M. Farach-Colton. The LCA problem revisited. LATIN 2000, LNCS 1776, 88–94.

[6] J. Fischer, V. Heun. Theoretical and practical improvements on the RMQ-problem, with applications to LCA and LCE. CPM 2006, LNCS 4009, 36–48.

[7] J. Fischer. Optimal succinctness for range minimum queries. LATIN 2010, LNCS 6034, 158–169.

[8] J. Fischer, V. Heun. Space-efficient preprocessing schemes for range minimum queries on static arrays. SIAM J. Comput. 40(2):465–492, 2011.

[9] K. Sadakane. Succinct data structures for flexible text retrieval systems. J. Discrete Algorithms 5(1):12–22, 2007.

[10] G. S. Brodal, P. Davoodi, S. S. Rao. On space efficient two dimensional range minimum data structures. Algorithmica 63(4):815–830, 2012.

[11] M. Liu, H. Yu. Lower bound for succinct range minimum query. STOC 2020, 1402–1415.

[12] S. Alstrup, T. Husfeldt, T. Rauhe. Marked ancestor problems. FOCS 1998, 534–543.

[13] E. D. Demaine, G. M. Landau, O. Weimann. On Cartesian trees and range minimum queries. Algorithmica 68(3):610–625, 2014.

[14] A. Amir, J. Fischer, M. Lewenstein. Two-dimensional range minimum queries. CPM 2007, LNCS 4580, 286–294.

Thực nghiệm và ứng dụng

[15] H. Ferrada, G. Navarro. Improved range minimum queries. J. Discrete Algorithms 43:72–80, 2017.

[16] N. Baumstark, S. Gog, T. Heuer, J. Labeit. Practical range minimum queries revisited. SEA 2017, LIPIcs 75, art. 12.

[17] T. Kowalski, S. Grabowski. Faster range minimum queries. Softw. Pract. Exp. 48:2043–2060, 2018.

[18] M. Dima, R. Ceterchi. Efficient range minimum queries using binary indexed trees. Olympiads in Informatics 9:39–44, 2015.

[19] M. Alzamel, P. Charalampopoulos, C. S. Iliopoulos, S. P. Pissis. How to answer a small batch of RMQs or LCA queries in practice. IWOCA 2017, LNCS 10765, 343–355.

[20] Meneses, Navarro, Ferrada, Quezada. Accelerating range minimum queries with ray tracing cores. Future Gener. Comput. Syst. 157:98–111, 2024.

[21] S. Muthukrishnan. Efficient algorithms for document retrieval problems. SODA 2002, 657–666.

[22] G. Navarro. Spaces, trees, and colors: the algorithmic landscape of document retrieval on sequences. ACM Comput. Surv. 46(4):52, 2014.

[23] L. Ilie, G. Navarro, L. Tinta. The longest common extension problem revisited and applications to approximate string searching. J. Discrete Algorithms 8(4):418–428, 2010.

[24] C. M. Zmasek, S. R. Eddy. A simple algorithm to infer gene duplication and speciation events on a gene tree. Bioinformatics 17(9):821–828, 2001.

[25] D. Lemire. Streaming maximum-minimum filter using no more than three comparisons per element. Nordic J. Computing 13(4):328–339, 2006.

[26] C.-T. Ho, R. Agrawal, N. Megiddo, R. Srikant. Range queries in OLAP data cubes. SIGMOD 1997, 73–88.

[27] brunomont. Range minimum query in $O(1)$ with linear time construction. Codeforces blog 78931, 2020.

[28] Học liệu: S. Wild, COMP526 Unit 9 (video); Stanford CS166 RMQ; VNOI Wiki (RMQ, LCA); cp-algorithms.

CẢM ƠN THẦY CÔ VÀ CÁC BẠN

Hỏi & Đáp

Mã nguồn, benchmark, video manim và báo cáo nghiên cứu: thư mục `seminar/`

$\langle O(1), O(n) \rangle$ $\langle O(n), O(\log n) \rangle$ $\langle O(n \log n), O(1) \rangle$ $\langle O(n), O(1) \rangle$

Chuẩn bị trả lời

Truy vấn $O(1)$ khi phải tính log?
Bảng lg[] $O(n)$ hoặc `__builtin_clz`; tránh log2() số thực.

Có cập nhật thì Sparse Table?
Một cập nhật làm hỏng $\Theta(n)$ ô ⇒ xây lại; dùng Segment Tree (cận dưới $\Theta(\log n / \log\log n)$).

Giá trị trùng nhau?
So sánh cặp (A[i], i), chọn trái nhất, nhất quán ở mọi bước.

Fischer–Heun hơn Sparse Table ở đâu?
Build và bộ nhớ $O(n)$ thay vì $O(n \log n)$; đổi lại hằng số và độ phức tạp cài đặt.

$O(n)$ bộ nhớ có phải $O(n)$ bit?
Không: $O(n)$ word = $O(n \log n)$ bit. $2n + o(n)$ bit là kết quả succinct riêng, chỉ trả argmin.

Bitmask RMQ dùng cho gcd được không?
Không: cần đáp án là một phần tử và phép so sánh (min/max). gcd dùng Sparse Table.

Ai đạt $\langle O(n), O(1) \rangle$ đầu tiên?
Gabow–Bentley–Tarjan 1984 (qua LCA của Harel–Tarjan), không phải Bender–Farach-Colton 2000.

Euler tour dài bao nhiêu?
2n − 1 phần tử (ghi nút khi đi xuống và mỗi lần quay về).

Lazy Segment Tree: cộng / gán cả đoạn

Nếu cả nút được cộng c thì min của nút cũng tăng c ⇒ lưu **tag** tại nút, chỉ đẩy xuống khi cần đi sâu hơn.

Mỗi thao tác chạm $O(\log n)$ nút ⇒ $O(\log n)$

Gán đoạn: tag gán phải phân biệt "chưa có tag" với "gán bằng 0"; gộp tag gán rồi cộng đúng thứ tự.

VÍ DỤ (TỪ MẢNG GỐC)

`A = [5, 2, 6, 3, 1, 7, 4, 8]`

`+3 vào [2, 5] → [5, 5, 9, 6, 4, 7, 4, 8]`

min [2, 7] = **4** (trước đó là 1)

`gán [2, 5] := 10 → [5, 10, 10, 10, 10, 7, 4, 8]`

min [2, 7] = **4**

Ma trận: hình chữ nhật nhỏ nhất

SPARSE TABLE 2D

$st[a][b][i][j]$ = min hình chữ nhật $2^a \times 2^b$; truy vấn ghép 4 hình chồng lấn ⇒ $O(1)$.

Build/bộ nhớ $O(RC \log R \log C)$. Ví dụ: ma trận [[5,2,6], [3,1,7],[4,8,0]], hàng 1–2 × cột 2–3 → 1.

Bài tập: CodeChef CHSQARR; VNOI Wiki có mục Sparse Table 2D.

INDEXING

$O(N)$ bit thêm, $O(1)$ truy vấn, $O(N)$ tiền xử lý; N/c bit ⇒ $\Omega(c)$ (Brodal–Davoodi–Rao 2010/12).

ENCODING

Không có “cây Cartesian 2D”: số mẫu đáp án quá lớn ⇒ cần $\Omega(N \log m)$ bit (Demaine–Landau–Weimann 2009/14; Brodal–Brodnik–Davoodi 2013).

Khi biết trước truy vấn, hoặc phép toán không lũy đẳng

ARPA'S TRICK (OFFLINE)

- Sắp truy vấn theo r; quét i = 1..n với ngăn xếp đơn điệu.
- Phần tử bị pop được hợp (DSU) vào i.
- Truy vấn (l, r) tại r: đáp án A[find(l)].
- Tổng $O((n+q)\cdot\alpha(n))$.

Lô nhỏ q ≪ n: co các vùng giữa đầu mút thành min của vùng, giữ ánh xạ chỉ số (Alzamel et al. 2017).

DISJOINT SPARSE TABLE

- Mỗi tầng chia mảng thành khối 2^k; lưu tổng hậu tố/tiền tố từ điểm giữa khối.
- Truy vấn: tầng = bit cao nhất của $l \oplus r$; đáp án = suf[l] ⊕ pre[r].
- $O(n \log n)$ build, $O(1)$ truy vấn, đúng cho **mọi phép kết hợp**.

Sqrt Tree: $O(n \log\log n)$, $O(1)$ truy vấn, hỗ trợ cập nhật (cp-algorithms).

PHỤ LỤC E · TÁI LẬP KẾT QUẢ

Chạy lại benchmark, hình và video

```
# Benchmark (≈ 1 phút): tự kiểm tra tính đúng rồi đo 4 thí nghiệm
cd seminar/benchmark
g++ -O3 -DNDEBUG -std=c++17 -march=native rmq_bench.cpp -o rmq_bench
taskset -c 2 ./rmq_bench all > results_raw.csv 2> run.log

# Biểu đồ (matplotlib) → slides/figures/*.svg
python seminar/figures/make_figures.py

# Video manim (3b1b/manim, ManimGL) → slides/media/*.mp4
cd seminar/manim && ./render_all.sh --hd ../slides/media

# Trình chiếu
cd seminar/slides && python -m http.server 8000   # mở http://localhost:8000
```

Kết quả sẽ khác theo máy/trình biên dịch/tải hệ thống. Biên dịch trên chính máy chạy (vì -march=native).